Internet Applications

Express Routing Fundamentals

Suhel Hammoud

What Will We Learn?

- Choose a method and URL for an action.
- Read an ID from a route parameter.
- Filter a list using a query string.
- Create, update, and delete assignments in an array.
- Understand why the order of routes matters.

A Route Matches a Method and a Path

Remember our health route:

```
app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});
```

- The method is `GET`.
- The path is `/health`.
- The function handles a matching request.

Today, our routes will work with assignments instead.

Name the Resource in the URL

An assignment is a **resource**: something our API works with.

```
/assignments       A collection of assignments
/assignments/1     One assignment with ID 1
```

- Use a plural noun for the collection: `/assignments`.
- Use the HTTP method to describe the action.
- For example, use `DELETE /assignments/1` to remove assignment 1.
- Avoid putting action names such as `/deleteAssignment` in the URL.

Our Assignment Routes

Method	Path	Action
GET	/assignments	List assignments
GET	/assignments/:id	Read one assignment
POST	/assignments	Create an assignment
PUT	/assignments/:id	Replace its editable fields
PATCH	/assignments/:id	Change part of it
DELETE	/assignments/:id	Remove an assignment

We will build these routes one at a time.

Start from the Previous Project

Use the Express 5 project and TypeScript settings from the last lesson.

Replace `src/index.ts` with this starting point:

```ts
import express from "express";

const app = express();
app.use(express.json());

// Add the data and routes here.

app.listen(3000, () => {
  console.log("Server: http://localhost:3000");
});
```

`express.json()` will let our routes read JSON request bodies.

Describe One Assignment

Below `app.use(express.json())`, add:

```ts
interface Assignment {
  id: number;
  title: string;
  completed: boolean;
}
```

- `id` identifies the assignment.
- `title` describes the work.
- `completed` is either `true` or `false`.

Keep the Data in an Array

Below the interface, add:

```
let assignments: Assignment[] = [
  { id: 1, title: "Practice types", completed: false },
  { id: 2, title: "Build /health", completed: true },
];

let nextId = 3;
```

- The next assignment we create will get ID 3.
- This is **in-memory** data: it lives inside the running program.
- Restarting the server restores this starting array.
- We will add a database in a later lesson.

GET: List All Assignments

Add this route below the data, before `app.listen()`:

```
app.get("/assignments", (req, res) => {
  res.json(assignments);
});
```

- The client asks for the collection.
- We send the array as JSON.
- The response uses status `200` by default.

Try the List Route

In terminal 1, check and start the app:

```
npx tsc --noEmit
npx tsx src/index.ts
```

In terminal 2:

```
curl -i http://localhost:3000/assignments
```

- Look for 200 OK and an array containing two assignments.
- Keep terminal 1 running while sending requests in terminal 2.
- After editing code, stop with Ctrl+C, check, and start again.

Route Parameters: Which Assignment?

```
Route:    /assignments/:id
Request:  /assignments/1
Value:    req.params.id is "1"
```

- `:id` captures a value from that part of the URL.
- The name `id` becomes a property on `req.params`.
- The captured value is a string, even when it looks like a number.
- `Number(req.params.id)` converts it for comparison with our IDs.

A JavaScript Reminder: find()

```
const item = assignments.find((assignment) => {
  return assignment.id === 1;
});
```

- `find()` looks for the first item that passes the test.
- Here, the test checks whether an assignment's ID is `1`.
- It returns the object, or `undefined` if no item matches.

The shorter form means the same thing:

```
const item = assignments.find((a) => a.id === 1);
```

GET: Read One Assignment

Add this route after the list route:

```
app.get("/assignments/:id", (req, res) => {
  const id = Number(req.params.id);
  const item = assignments.find((a) => a.id === id);

  if (item === undefined) {
    res.status(404).json({ error: "Assignment not found" });
    return;
  }

  res.json(item);
});
```

- A missing assignment gets `404 Not Found`.
- `return` stops this handler after sending the error response.

Try an Existing and a Missing ID

```
curl -i http://localhost:3000/assignments/1
curl -i http://localhost:3000/assignments/999
```

- ID 1: status 200 and one assignment object.
- ID 999: status 404 and an error message.
- :id is a placeholder in the route definition.
- In the request URL, replace it with the actual ID.

Query Strings: Which Items in the List?

```
/assignments?completed=true
```

- The path is still `/assignments`.
- The query string starts after `?`.
- `req.query.completed` gives us the filter value.
- In this request, that value is the string `"true"`.

Use a parameter to identify an item; use a query to filter a collection.

GET: Filter the List

Replace the original `GET /assignments` handler with this one:

```
app.get("/assignments", (req, res) => {
  const completed = req.query.completed;
  if (completed === undefined) {
    res.json(assignments);
    return;
  }
  if (completed !== "true" && completed !== "false") {
    res.status(400).json({ error: "Use true or false" });
    return;
  }
  const filtered = assignments.filter((item) => {
    return item.completed === (completed === "true");
  });
  res.json(filtered);
});
```

Understand and Try the Filter

- `filter()` returns a new array containing matching items.
- `completed === "true"` converts our allowed string to a boolean.
- Do not use `Boolean("false")`: a nonempty string becomes `true`.

```
curl -i 'http://localhost:3000/assignments?completed=true'
curl -i 'http://localhost:3000/assignments?completed=false'
curl -i 'http://localhost:3000/assignments?completed=maybe'
```

- `true`: returns only completed assignments.
- `false`: returns only incomplete assignments.
- `maybe`: returns `400 Bad Request` for an invalid filter.

The Same Path Can Have Different Methods

```
app.get("/assignments", (req, res) => {
  // Read the list.
});

app.post("/assignments", (req, res) => {
  // Create a new item.
});
```

These are examples of the method names; keep your actual handlers.

The path is the same, but the method selects a different action.

Send New Data in the Request Body

For a new assignment, the client sends:

```
{"title":"Practice routing"}
```

- The JSON body carries the data to create.
- The client sets `Content-Type: application/json`.
- `express.json()` makes parsed JSON available through `req.body`.
- Register that parser before the routes that need it.

The server will choose the ID and set `completed` to `false`.

POST: Create an Assignment

Add this route before app.listen():

```
app.post("/assignments", (req, res) => {
  const title = req.body?.title;
  if (typeof title !== "string" || title.trim() === "") {
    res.status(400).json({ error: "A title is required" });
    return;
  }
  const item: Assignment = {
    id: nextId++,
    title: title.trim(),
    completed: false,
  };
  assignments.push(item);
  res.status(201).json(item);
});
```

nextId++ uses the current ID, then increases it for the next item.

Try Creating an Assignment

```
curl -i -X POST http://localhost:3000/assignments \
  -H 'Content-Type: application/json' \
  -d '{"title":"Practice routing"}'
```

- `-X POST` chooses the method.
- `-H` adds a request header; `-d` supplies the body.
- Expect `201 Created` and the new assignment with ID `3`.

```
curl -i http://localhost:3000/assignments/3
```

The new assignment stays available until the server restarts.

PUT and PATCH: Two Ways to Update

For this API, an assignment has two editable fields:

```
{"title":"Practice routing","completed":false}
```

- PUT replaces both editable fields; send both values.
- PATCH changes only part of the assignment.
- Our small PATCH route will change only completed.
- The ID stays the same in both cases.
- Our update routes return 404 when the ID does not exist.

PUT: Replace the Editable Fields

Add this route before app.listen():

```
app.put("/assignments/:id", (req, res) => {
  const id = Number(req.params.id);
  const item = assignments.find((a) => a.id === id);
  if (item === undefined) {
    res.status(404).json({ error: "Assignment not found" });
    return;
  }
  const title = req.body?.title;
  const completed = req.body?.completed;
  if (typeof title !== "string" || title.trim() === "" ||
      typeof completed !== "boolean") {
    res.status(400).json({ error: "Send title and completed" });
    return;
  }
  item.title = title.trim();
  item.completed = completed;
  res.json(item);
});
```

Try Replacing an Assignment's Fields

```
curl -i -X PUT http://localhost:3000/assignments/1 \
  -H 'Content-Type: application/json' \
  -d '{"title":"Review TypeScript","completed":true}'
```

Expected body:

```
{"id":1,"title":"Review TypeScript","completed":true}
```

- Both editable fields now have the supplied values.
- Omitting either field gives 400 in our API.
- The server keeps the assignment's ID.

PATCH: Change Only completed

Add this route before `app.listen()`:

```
app.patch("/assignments/:id", (req, res) => {
  const id = Number(req.params.id);
  const item = assignments.find((a) => a.id === id);
  if (item === undefined) {
    res.status(404).json({ error: "Assignment not found" });
    return;
  }
  const completed = req.body?.completed;
  if (typeof completed !== "boolean") {
    res.status(400).json({ error: "Send a completed boolean" });
    return;
  }
  item.completed = completed;
  res.json(item);
});
```

The title stays as it was. This route only updates `completed`.

Try a Partial Update

```
curl -i -X PATCH http://localhost:3000/assignments/1 \
  -H 'Content-Type: application/json' \
  -d '{"completed":false}'
```

- Expect status `200` and the updated assignment.
- Only `completed` changes; there is no need to send a title.
- Send a JSON boolean: `false`, not the string `"false"`.

We use small checks here. We will study validation in a later lesson.

DELETE: Remove an Assignment

Add this route before `app.listen()`:

```
app.delete("/assignments/:id", (req, res) => {
  const id = Number(req.params.id);
  const item = assignments.find((a) => a.id === id);
  if (item === undefined) {
    res.status(404).json({ error: "Assignment not found" });
    return;
  }
  assignments = assignments.filter((a) => a.id !== id);
  res.status(204).end();
});
```

- `filter()` keeps every assignment except the one being removed.
- `204 No Content` means success with no response body.

Try Deleting an Assignment

```
curl -i -X DELETE http://localhost:3000/assignments/1
curl -i http://localhost:3000/assignments/1
```

- The first request returns `204` with an empty body.
- The second returns `404` because assignment 1 is gone.
- Sending DELETE for the same ID again also returns `404` here.
- `nextId` continues increasing; deleting does not reuse an ID.

Route Order: A Possible Surprise

Imagine adding `GET /assignments/summary` below our ID route:

```
First:   GET /assignments/:id
Later:   GET /assignments/summary
```

What happens when we request `/assignments/summary`?

- `:id` also matches the word `summary`.
- Our ID handler runs first and sends `404` for the failed lookup.
- The summary handler never runs.
- Express does not automatically choose the most specific route.

🟦 Register the Named Route First

Add this route before `GET /assignments/:id`:

```
app.get("/assignments/summary", (req, res) => {
  res.json({ count: assignments.length });
});
```

The order should now be:

```
GET /assignments
GET /assignments/summary
GET /assignments/:id
```

Call `/assignments/summary` to see the current number of items.

Put the Pieces Together

Your file should have this order:

```
1. Import Express and create app.
2. Register express.json().
3. Define Assignment, assignments, and nextId.
4. GET /assignments             (the filtered version)
5. GET /assignments/summary
6. GET /assignments/:id
7. POST /assignments
8. PUT /assignments/:id
9. PATCH /assignments/:id
10. DELETE /assignments/:id
11. app.listen(...)
```

Keep one handler for each method and path in this exercise.

Practice: Run the Whole Sequence

Finish adding all the routes, then check and restart the server once.

Use the earlier `curl` examples in this order:

1. GET the list: expect the two starting assignments.
2. POST a new assignment: expect ID `3` and status `201`.
3. GET `/assignments/3`: expect the new assignment.
4. PUT `/assignments/1`: replace its title and completed value.
5. PATCH `/assignments/1`: change only its completed value.
6. DELETE `/assignments/1`, then GET it: expect `204`, then `404`.

Keep the server running throughout so its data is preserved.

Practice: Check Filtering and Order

With that same server still running:

```
curl -i 'http://localhost:3000/assignments?completed=true'
curl -i http://localhost:3000/assignments/summary
curl -i http://localhost:3000/assignments/999
```

- The filtered list should contain assignment `2`.
- The summary should contain `{"count":2}`: IDs `2` and `3` remain.
- ID `999` should return `404`.

Restart the server and GET the list again. What changed, and why?

References

Routing (https://expressjs.com/en/guide/routing/)

Express API (https://expressjs.com/en/api/)

HTTP methods (https://www.rfc-editor.org/rfc/rfc9110.html)